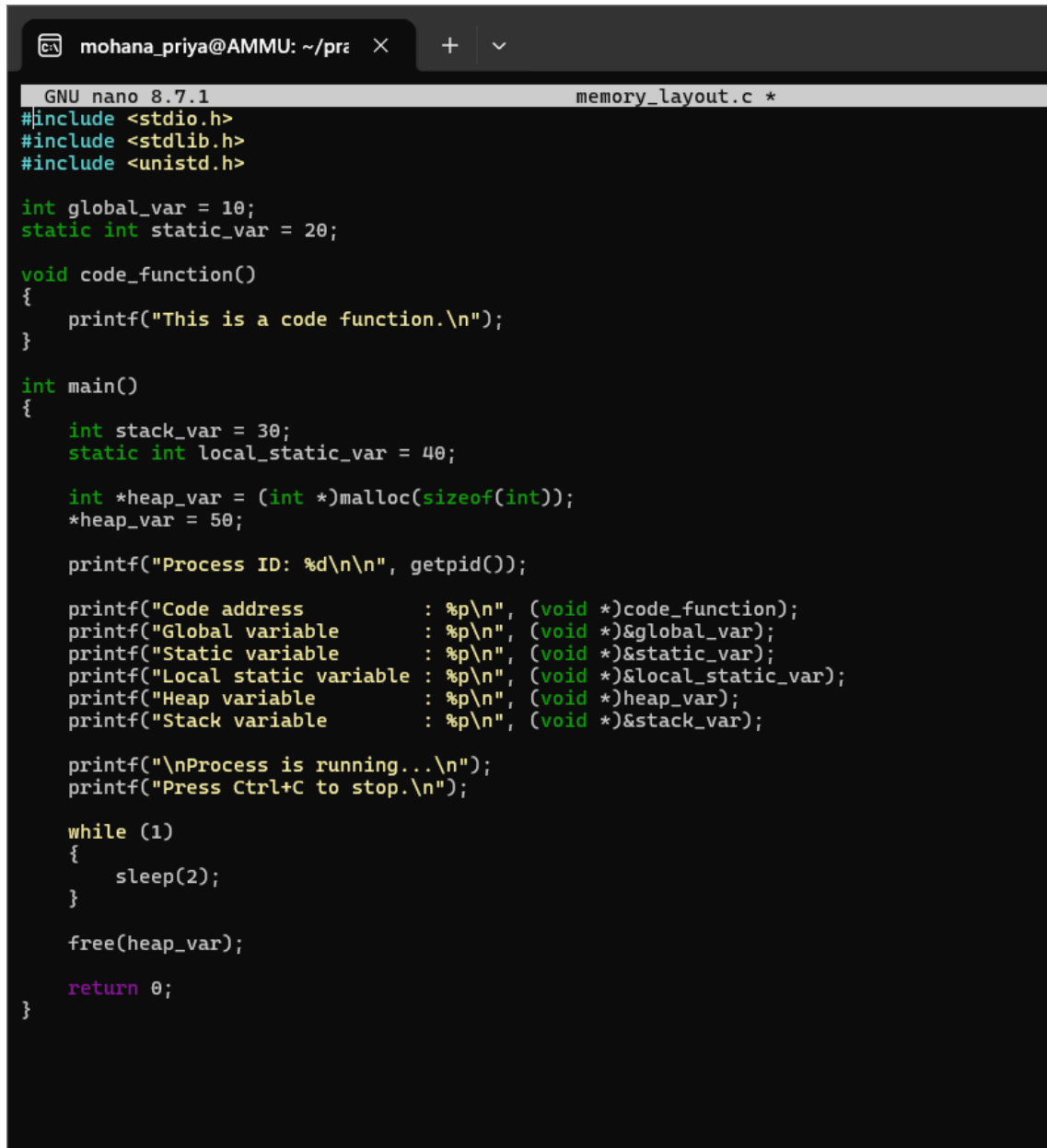


Practical – 7

Task –1 Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.



The screenshot shows a terminal window with a dark background. At the top, there's a tab labeled 'mohana_priya@AMMU: ~/pr' with a close button and a dropdown arrow. Below the tab, the terminal title bar reads 'GNU nano 8.7.1' and the filename 'memory_layout.c *'. The code is written in a syntax-highlighted C language. It includes headers for stdio, stdlib, and unistd. It defines a global variable, a static variable, a code function, and a main function. The main function prints the addresses of these variables and a heap-allocated memory block. It also includes a while loop with a sleep function and a free function call.

```
GNU nano 8.7.1 memory_layout.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 10;
static int static_var = 20;

void code_function()
{
    printf("This is a code function.\n");
}

int main()
{
    int stack_var = 30;
    static int local_static_var = 40;

    int *heap_var = (int *)malloc(sizeof(int));
    *heap_var = 50;

    printf("Process ID: %d\n\n", getpid());

    printf("Code address      : %p\n", (void *)code_function);
    printf("Global variable   : %p\n", (void *)&global_var);
    printf("Static variable    : %p\n", (void *)&static_var);
    printf("Local static variable : %p\n", (void *)&local_static_var);
    printf("Heap variable      : %p\n", (void *)heap_var);
    printf("Stack variable     : %p\n", (void *)&stack_var);

    printf("\nProcess is running...\n");
    printf("Press Ctrl+C to stop.\n");

    while (1)
    {
        sleep(2);
    }

    free(heap_var);

    return 0;
}
```

```
mohana_priya@AMMU:~$ mkdir practical_memory
mohana_priya@AMMU:~$ cd practical_memory
mohana_priya@AMMU:~/practical_memory$ nano memory_layout.c
mohana_priya@AMMU:~/practical_memory$ gcc memory_layout.c -o memory_layout
mohana_priya@AMMU:~/practical_memory$ ./memory_layout
Process ID: 757

Code address      : 0x55c67c7d31c9
Global variable   : 0x55c67c7d6010
Static variable   : 0x55c67c7d6014
Local static variable : 0x55c67c7d6018
Heap variable     : 0x55c6a55f9010
Stack variable    : 0x7ffc49ae650c

Process is running...
Press Ctrl+C to stop.
^C
mohana_priya@AMMU:~/practical_memory$
```

Task-2 Make use of the /proc//maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 10;
static int static_var = 20;

void display()
{
    printf("Inside code segment\n");
}

int main()
{
    int stack_var = 30;
    static int local_static = 40;

    int *heap_var = malloc(sizeof(int));
    *heap_var = 50;

    printf("Process ID: %d\n", getpid());

    printf("Code address      : %p\n", (void *)display);
    printf("Global variable    : %p\n", (void *)&global_var);
    printf("Static variable      : %p\n", (void *)&static_var);
    printf("Local static variable : %p\n", (void *)&local_static);
    printf("Heap variable        : %p\n", (void *)heap_var);
    printf("Stack variable       : %p\n", (void *)&stack_var);

    printf("\nProcess is running...\n");

    while (1)
    {
        sleep(2);
    }

    free(heap_var);

    return 0;
}
```

```
mohana_priya@AMMU: ~  
mohana_priya@AMMU:~$ nano memory_analysis.c  
mohana_priya@AMMU:~$ nano memory_analysis.c  
mohana_priya@AMMU:~$ ^C  
mohana_priya@AMMU:~$ gcc memory_analysis.c -o memory_analysis  
mohana_priya@AMMU:~$ ./memory_analysis &  
[1] 1017  
Process ID: 1017  
Code address      : 0x5d9c9ddb41c9  
Global variable   : 0x5d9c9ddb7010  
Static variable   : 0x5d9c9ddb7014  
Local static variable : 0x5d9c9ddb7018  
Heap variable     : 0x5d9cad30f010  
Stack variable    : 0x7ffde39b5ebc  
  
Process is running...  
mohana_priya@AMMU:~$ cat /proc/1017/maps  
5d9c9ddb3000-5d9c9ddb4000 r--p 00000000 08:30 2739 /home/mohana_priya/memory_analysis  
5d9c9ddb4000-5d9c9ddb5000 r-xp 00001000 08:30 2739 /home/mohana_priya/memory_analysis  
5d9c9ddb5000-5d9c9ddb6000 r--p 00002000 08:30 2739 /home/mohana_priya/memory_analysis  
5d9c9ddb6000-5d9c9ddb7000 r--p 00002000 08:30 2739 /home/mohana_priya/memory_analysis  
5d9c9ddb7000-5d9c9ddb8000 rw-p 00003000 08:30 2739 /home/mohana_priya/memory_analysis  
5d9cad30f000-5d9cad330000 rw-p 00000000 00:00 0 [heap]  
7a5d0c600000-7a5d0c628000 r--p 00000000 08:30 13628 /usr/lib/x86_64-linux-gnu/libc.so.6  
7a5d0c628000-7a5d0c7c0000 r-xp 00028000 08:30 13628 /usr/lib/x86_64-linux-gnu/libc.so.6  
7a5d0c7c0000-7a5d0c80e000 r--p 001c0000 08:30 13628 /usr/lib/x86_64-linux-gnu/libc.so.6  
7a5d0c80e000-7a5d0c812000 r--p 0020d000 08:30 13628 /usr/lib/x86_64-linux-gnu/libc.so.6  
7a5d0c812000-7a5d0c814000 rw-p 00211000 08:30 13628 /usr/lib/x86_64-linux-gnu/libc.so.6  
7a5d0c814000-7a5d0c821000 rw-p 00000000 00:00 0  
7a5d0c889000-7a5d0c88c000 rw-p 00000000 00:00 0  
7a5d0c891000-7a5d0c893000 rw-p 00000000 00:00 0  
7a5d0c893000-7a5d0c897000 r--p 00000000 00:00 0 [vvar]  
7a5d0c897000-7a5d0c899000 r--p 00000000 00:00 0 [vvar_vclock]  
7a5d0c899000-7a5d0c89b000 r-xp 00000000 00:00 0 [vdso]  
7a5d0c89b000-7a5d0c89c000 r--p 00000000 08:30 13625 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2  
7a5d0c89c000-7a5d0c8cb000 r-xp 00001000 08:30 13625 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2  
7a5d0c8cb000-7a5d0c8d6000 r--p 00030000 08:30 13625 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2  
7a5d0c8d6000-7a5d0c8d8000 r--p 0003b000 08:30 13625 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2  
7a5d0c8d8000-7a5d0c8d9000 rw-p 0003d000 08:30 13625 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2  
7a5d0c8d9000-7a5d0c8da000 rw-p 00000000 00:00 0  
7ffde3997000-7ffde39b8000 rw-p 00000000 00:00 0 [stack]  
mohana_priya@AMMU:~$ |
```